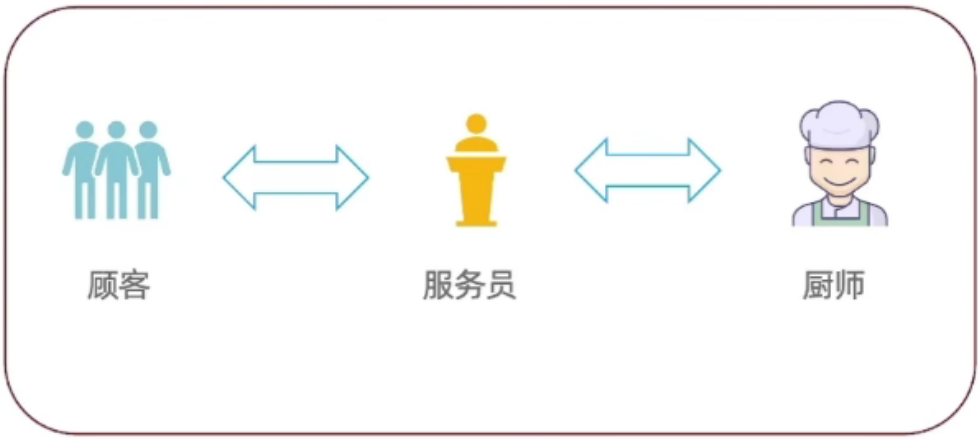
API接口

给前端和后端提供交互数据的通道。



用餐厅举例：

顾客（前端）：点菜（发送请求$Request$）

服务员（API）：传递菜单，端菜（接受请求，返回响应$Response$）

后厨（后端）：做菜（处理业务逻辑）

前端通过接口将请求传输给后端，后端处理请求并通过接口返回结果。

HTTP协议

Web通信的基础协议，客户端与服务器之间请求和应答的标准。（一种代码语言的规范标准）

URL

例：https://lingrui.studio/directions/#/problem/ML/3509169111223500800

`https`：通信协议

`lingrui.studio`：主机名，服务器地址

`directions/#/problem/ML/`：路径，资源位置

请求方法

`GET`：查看、获取数据

`POST`：提交、创建数据

`PUT`：更新、替换数据

`DELETE`：删除数据

请求头与请求体

用信封举例：

请求头：信封本身（寄件人、收件人、邮票、快递类型）

请求体：信纸的内容

状态码

服务器在响应请求时返回的一个三位数字，用来告诉客户端请求的结果。

`1xx`：信息性，请求已收到，继续处理

`2xx`：成功，请求成功

`3xx`：重定向，需要进一步操作

`4xx`：客户端错误，请求有语法错误或无法实现，比如404（┐(￣ヘ￣)┌）

`5xx`：服务器错误，服务器处理请求错误

响应头与响应体

响应头：响应的元信息（服务器类型、内容类型、设置Cookie等）

响应体：响应的实际数据（JSON、HTML文件）

发送请求：

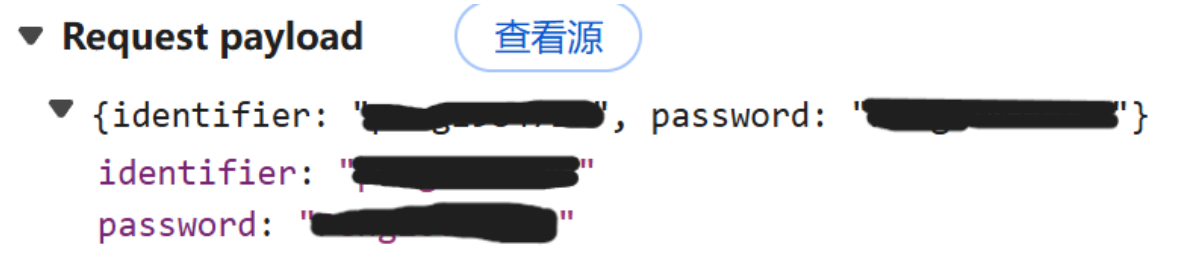
▼常规

请求 URL	https://recruitapi.lingrui.studio/auth/login
Request Method	POST
Status Code	200 OK
Remote Address	1.14.60.5:443
Referrer Policy	strict-origin-when-cross-origin

▶ 响应标头 (18)

▶ Request headers (19)

post中请求数据：



响应体：



接口测试

测试API本身，测试数据的交换、传递、状态管理是否正确。

Apifox

Query：

附在URL上的附加说明，用于GET请求，向服务器传递过滤、分页、排序等轻量级参数。

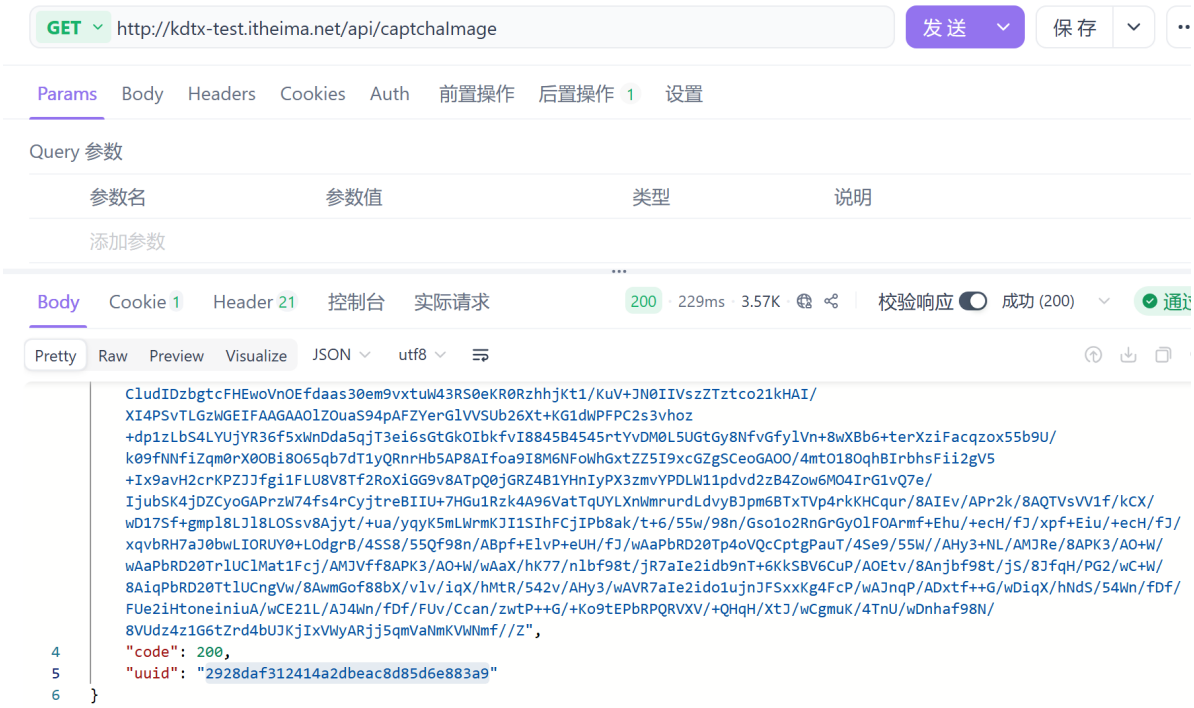
拼接在URL后，速度快，长度受限，适合简单数据。

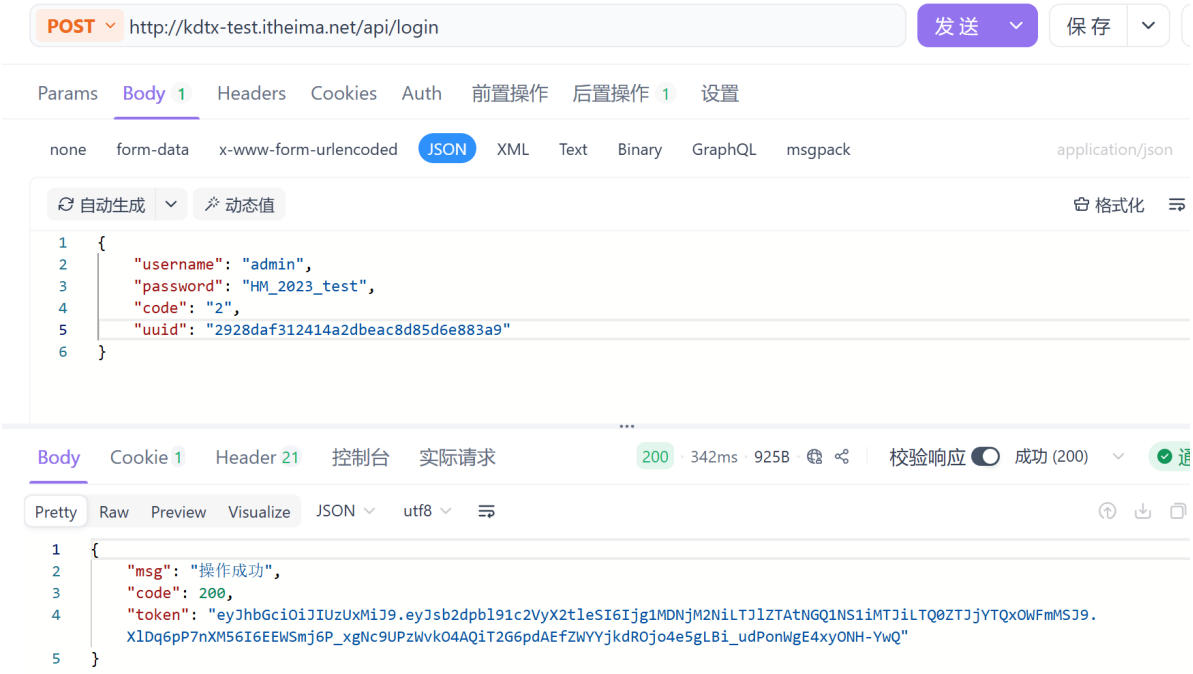
Content-Type：application/json（告诉服务器Body的格式）

Authorization：Bearer xxxx（身份认证令牌）

练习（获取图片验证码）

（题目和教程来自b站黑马程序员）





使用GET获取图象验证码uuid，再通过POST登录账号。

Fastapi

天生支持异步。

函数前增加`async`。

同步和异步：

同步：代码按顺序执行任务，会导致任务阻塞，运行速度慢。

异怖：遇到耗时操作时，同时经行其他任务，等耗时操作完成再回来继续，提高运行效率。

代码：

```
@app.get("/")
async def root():
    return {"message": "Hello world"}
```

装饰器：`@app.get("/")`

其中包含fastapi实例、请求方式和请求路径。

响应结果：`return {"message": "Hello world"}`

在终端中输入`uvicorn Fastapi:app --reload`启用fastapi，再访问根路径http://127.0.0.1:8000/



还可以更改“/”内容来更改路由。

参数：

使同一接口根据不同参数返回不同的数据，实现动态交互。

路径参数：

URL路径的一部分，用于指向唯一的，特定的资源。

使用`GET`。我们可以使用Path函数对其进行修饰。

```
@app.get("/book/{id}")
async def get_book(id: int = Path(..., gt = 0, lt = 101,
                                  description = "书籍的id，取值范围1~100")):
    return {"id": id, "title": f"这是第{id}本书"}
```

查询参数：

URL`?`之后，对资源集合进行搜索、过滤、排序或分页等。

使用`GET`。

```
@app.get("/news/news_list")
async def get_news_list(
    skip: int = Query(0, description = "跳过的记录数", lt = 100),
    limit: int = 10):
    return {"skip": skip, "limit": limit}
```

请求体：

HTTP请求的消息体（body），携带大量数据用于更新资源。

使用`POST`或`PUT`。

```
class User(BaseModel):
    username: str = Field(default = "dbw", min_length=2, max_length=10)
    password: str

@app.post("/register")
async def register(user: User):
    return user
```

calculate:

```
def op_sum(a: float, b: float) -> float:
    return a + b

def op_multiply(a: float, b: float) -> float:
    return a * b

def op_subtract(a: float, b: float) -> float:
    return a - b
```

```
def op_divide(a: float, b: float) -> float:
    if b == 0:
        raise ZeroDivisionError("除数不能为 0!")
    return a / b

OPS = {
    "sum": op_sum,
    "multiply": op_multiply,
    "subtract": op_subtract,
    "divide": op_divide,
}

def calculate(operation: str, a: float, b: float) -> float:
    if operation not in OPS:
        raise ValueError(f"不支持的操作: {operation}")
    return OPS[operation](a, b)
```

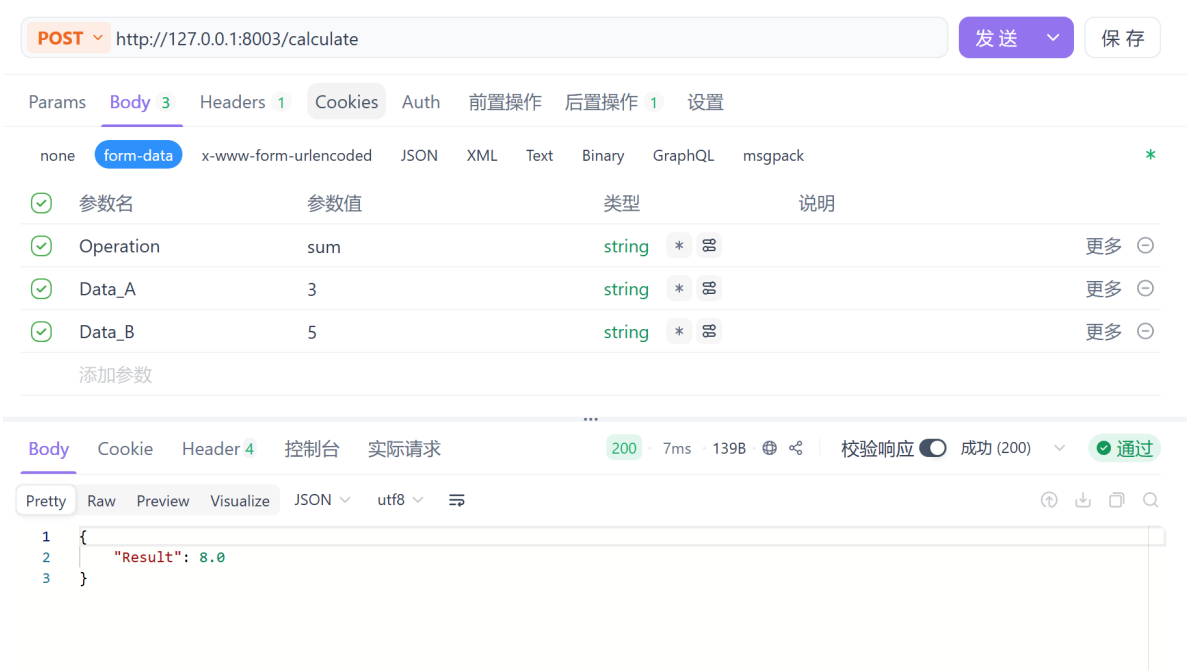
main:

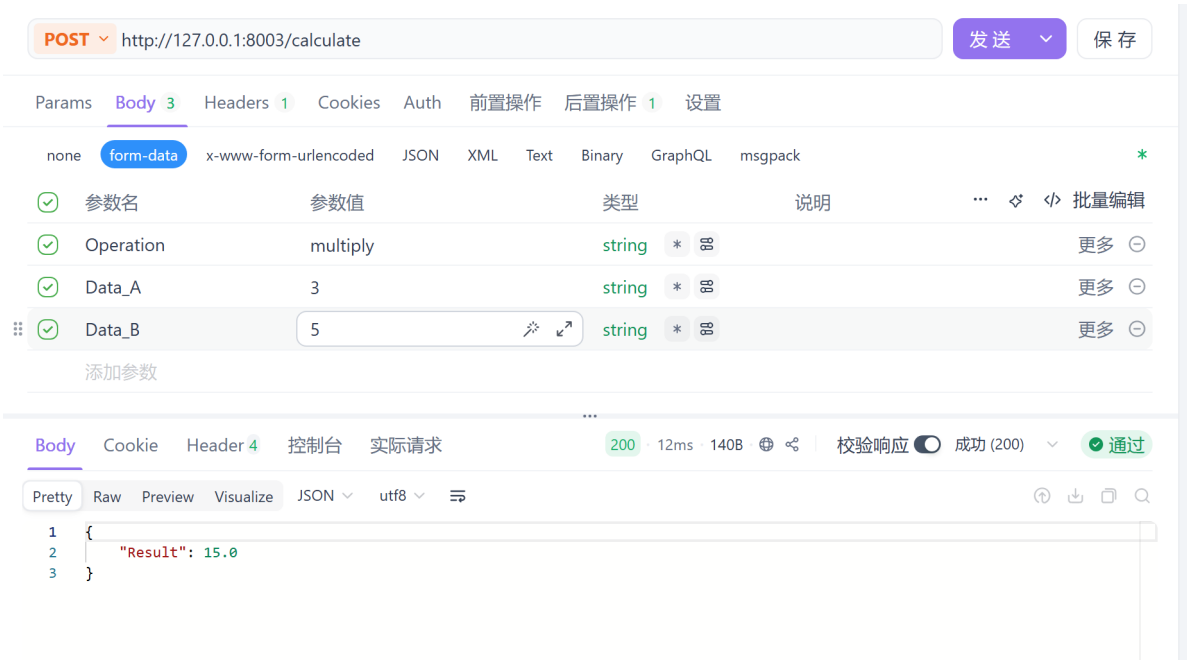
```
from fastapi import FastAPI, Form, HTTPException, Request, Header, File,
UploadFile
import calculator

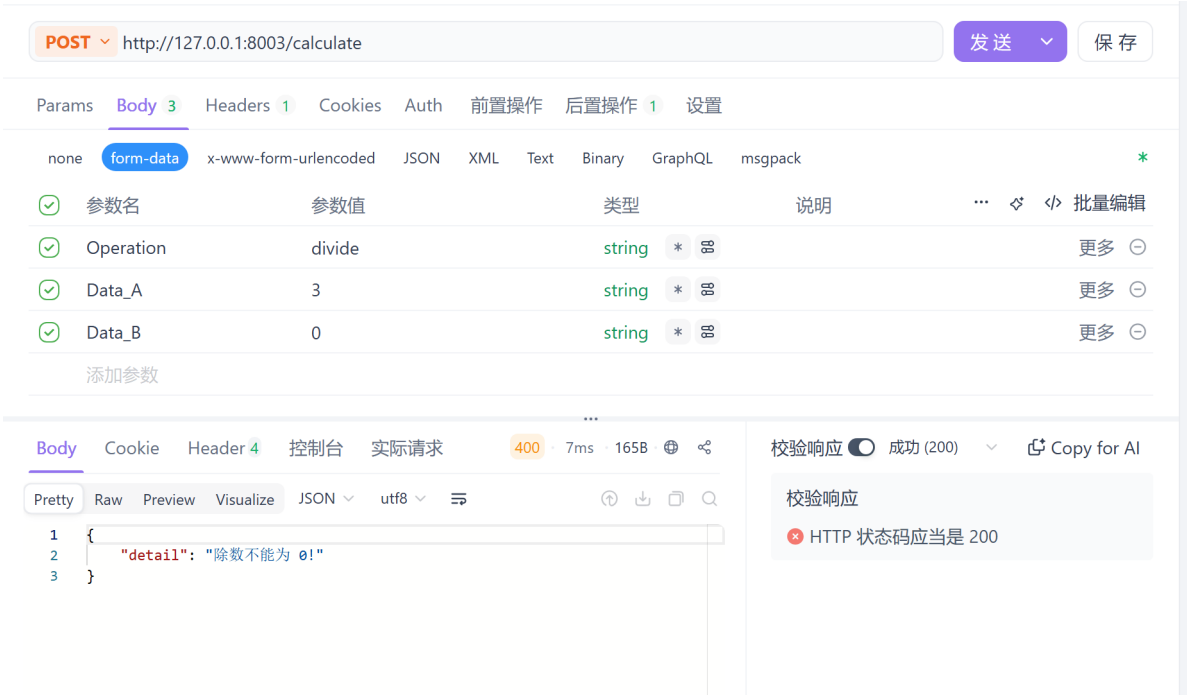
app = FastAPI()

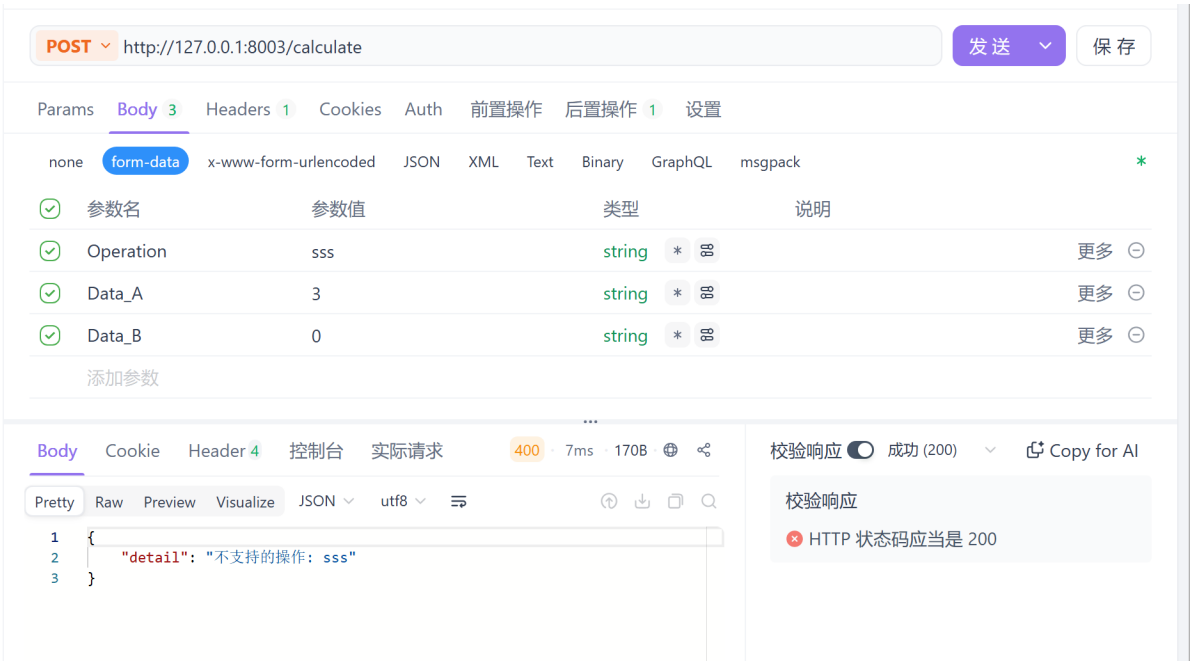
@app.post("/calculate")
async def calculate(
    request: Request,
    Operation: str = Form(...),
    Data_A: float = Form(...),
    Data_B: float = Form(...),
    x_token: str | None = Header(default = None),
):
    print("Content-Type:", request.headers.get("content-type"))
    print("X-Token:", x_token)
    try:
        result = calculator.calculate(Operation, Data_A, Data_B)
    except ZeroDivisionError as exc:
        raise HTTPException(status_code = 400, detail = str(exc))
    except ValueError as exc:
        raise HTTPException(status_code = 400, detail = str(exc))
    return {"Result": round(result, 4)}
```

最后apifox运行结果：


POST http://127.0.0.1:8003/calculate
发送
保存
Params Body 3 Headers 1 Cookies Auth 前置操作 后置操作 1 设置
none form-data x-www-form-urlencoded JSON XML Text Binary GraphQL msgpack
参数名 参数值 类型 说明
Operation sum string 更多
Data_A 3 string 更多
Data_B 5 string 更多
添加参数
Body Cookie Header 4 控制台 实际请求
200 7ms 139B
校验响应 成功 (200)
通过
Pretty Raw Preview Visualize JSON utf8
{
"Result": 8.0
}


POST http://127.0.0.1:8003/calculate
发送
保存
Params Body 3 Headers 1 Cookies Auth 前置操作 后置操作 1 设置
none form-data x-www-form-urlencoded JSON XML Text Binary GraphQL msgpack
参数名 参数值 类型 说明 批量编辑
Operation multiply string 更多
Data_A 3 string 更多
Data_B 5 string 更多
添加参数
Body Cookie Header 4 控制台 实际请求
200 12ms 140B
校验响应 成功 (200)
通过
Pretty Raw Preview Visualize JSON utf8
{
"Result": 15.0
}


POST http://127.0.0.1:8003/calculate
发送
保存
Params Body 3 Headers 1 Cookies Auth 前置操作 后置操作 1 设置
none form-data x-www-form-urlencoded JSON XML Text Binary GraphQL msgpack
参数名 参数值 类型 说明 批量编辑
Operation divide string 更多
Data_A 3 string 更多
Data_B 0 string 更多
添加参数
Body Cookie Header 4 控制台 实际请求
400 7ms 165B
校验响应 成功 (200)
Copy for AI
Pretty Raw Preview Visualize JSON utf8
{
"detail": "除数不能为 0!"
}
校验响应
HTTP 状态码应当是 200


POST
http://127.0.0.1:8003/calculate
发送
保存
Params
Body 3
Headers 1
Cookies
Auth
前置操作
后置操作 1
设置
none
form-data
x-www-form-urlencoded
JSON
XML
Text
Binary
GraphQL
msgpack
参数名
参数值
类型
说明
Operation
sss
string
更多
Data_A
3
string
更多
Data_B
0
string
更多
添加参数

Body
Cookie
Header 4
控制台
实际请求
400
7ms
170B
校验响应
成功 (200)
Copy for AI
Pretty
Raw
Preview
Visualize
JSON
utf8
1 {
2 "detail": "不支持的操作: sss"
3 }
校验响应
HTTP 状态码应当是 200